Lecture 8

Convolution Neural Network

Image Classification



100×100

Model

$$\begin{bmatrix} \vdots \\ 0.2 \\ 0.7 \\ 0.1 \\ \vdots \end{bmatrix} \quad y'$$

Cross entropy

$$\begin{matrix} \\ \text{dog} \\ \text{cat} \\ \text{tree} \\ \\ \end{matrix} \begin{bmatrix} \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \end{bmatrix} \quad \hat{y}$$

(All the images to be classified have the same size.)

Cross-Entropy

Intuitively Understanding the Cross Entropy

$$H(P^*|P) = -\sum_i \underbrace{P^*(i)}_{\text{TRUE CLASS DISTIRBUTION}} \log \underbrace{P(i)}_{\text{PREDICTED CLASS DISTIRBUTION}}$$

Intuitively Understanding the KL divergence

KL is not symmetric

$$D_{KL}(P||Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

$$p_1\log\frac{p_1}{q_1} + p_2\log\frac{p_2}{q_2}$$

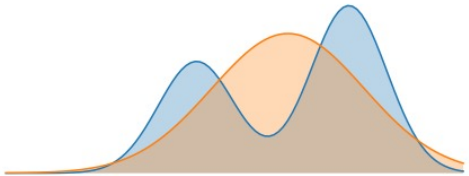
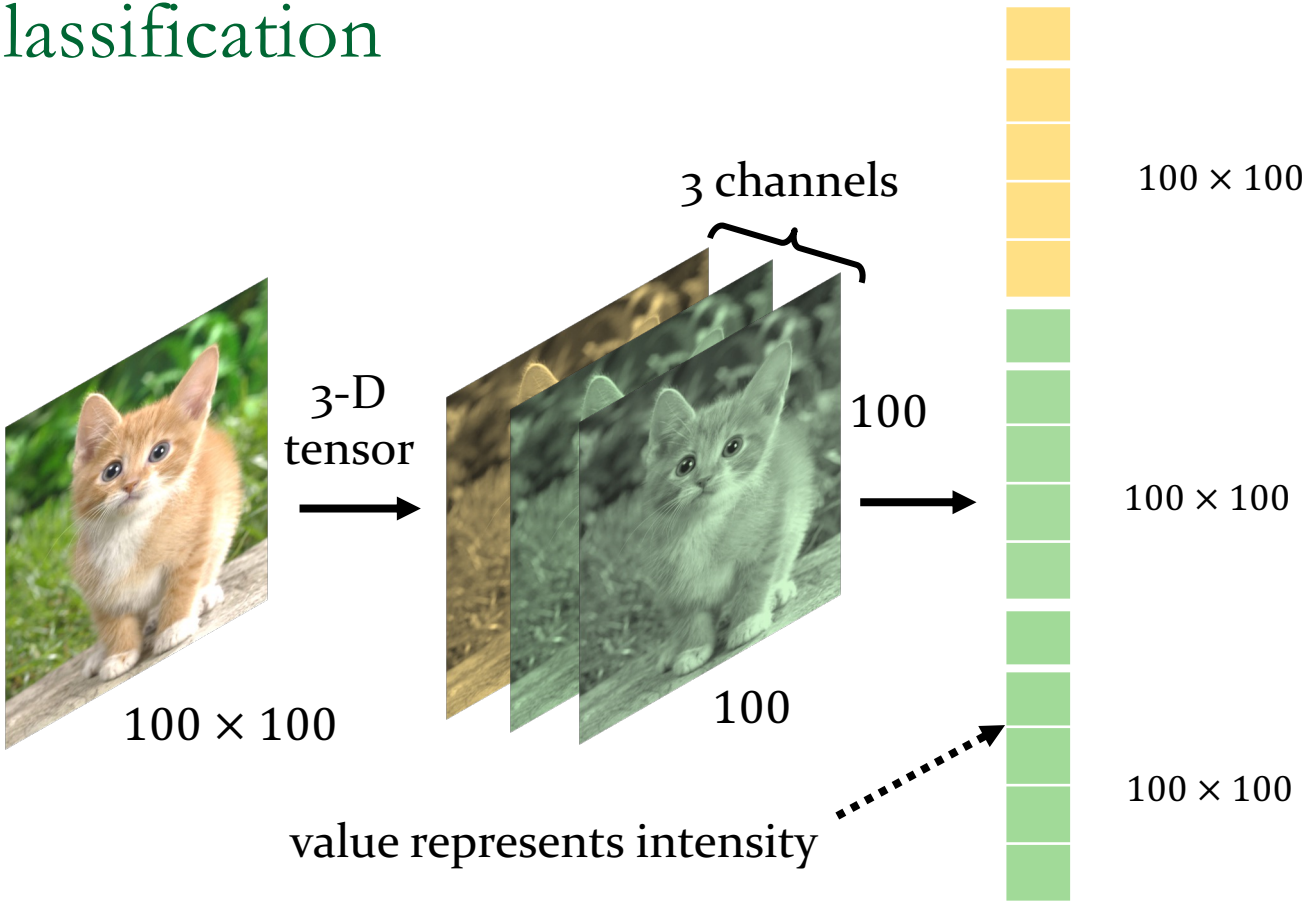
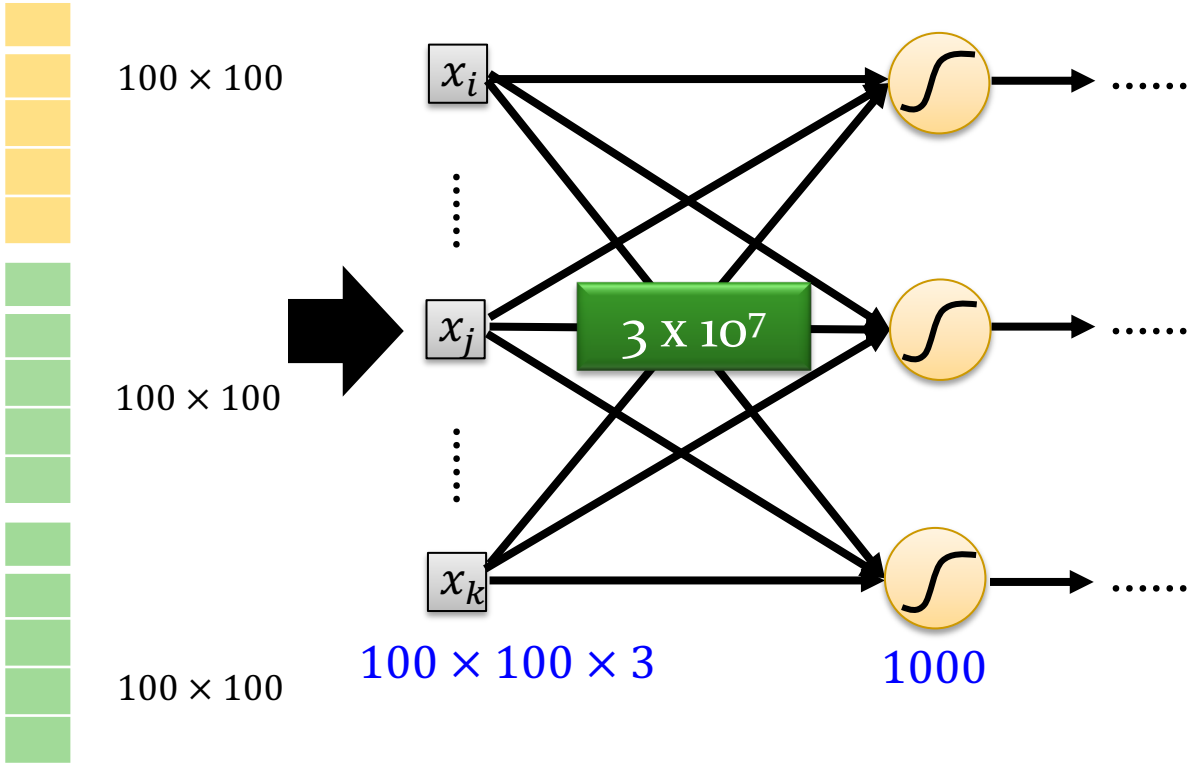


Image Classification

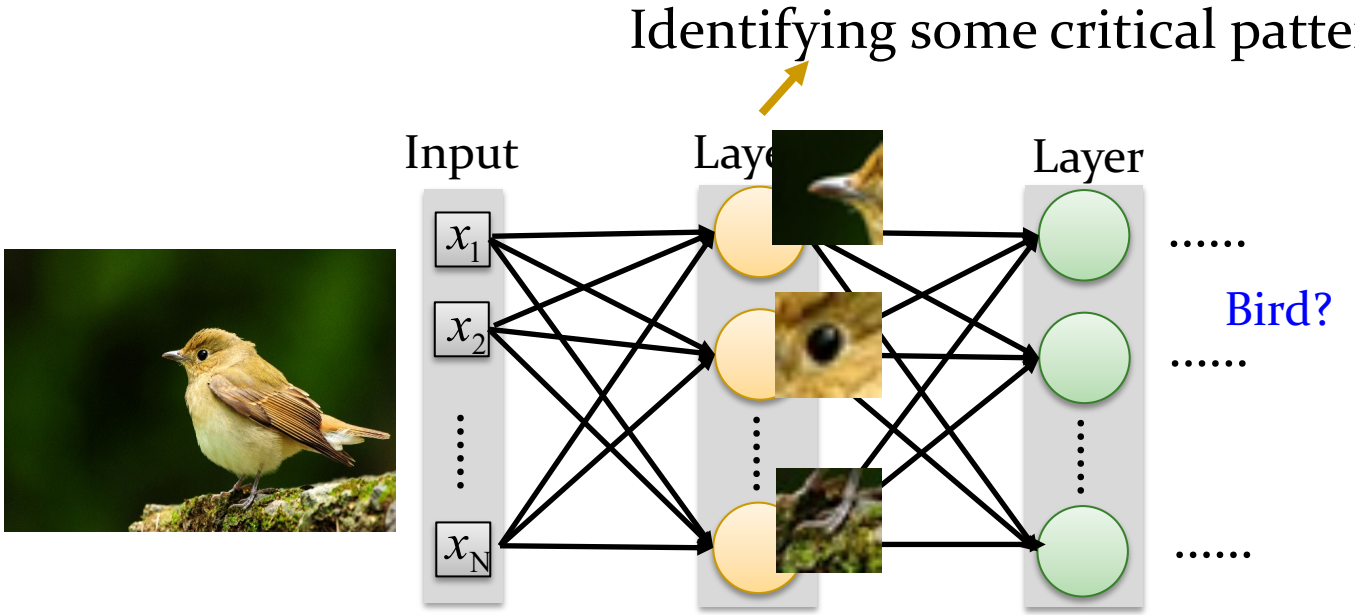


Fully Connected Network



Do we really need "fully connected" in image processing?

Observation 1

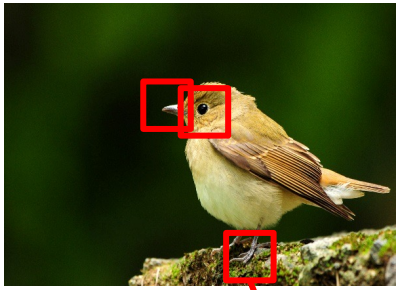


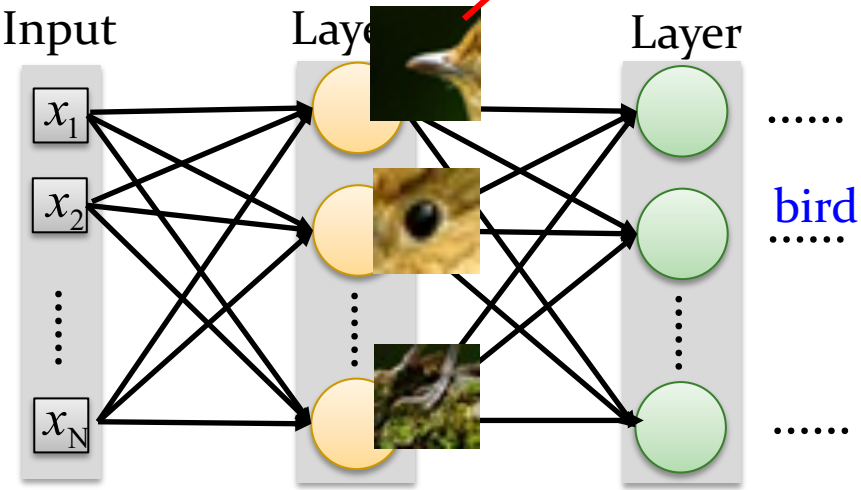
Perhaps human also identify birds in a similar way … ☺

Observation 1

A neuron does not have to see the whole image.

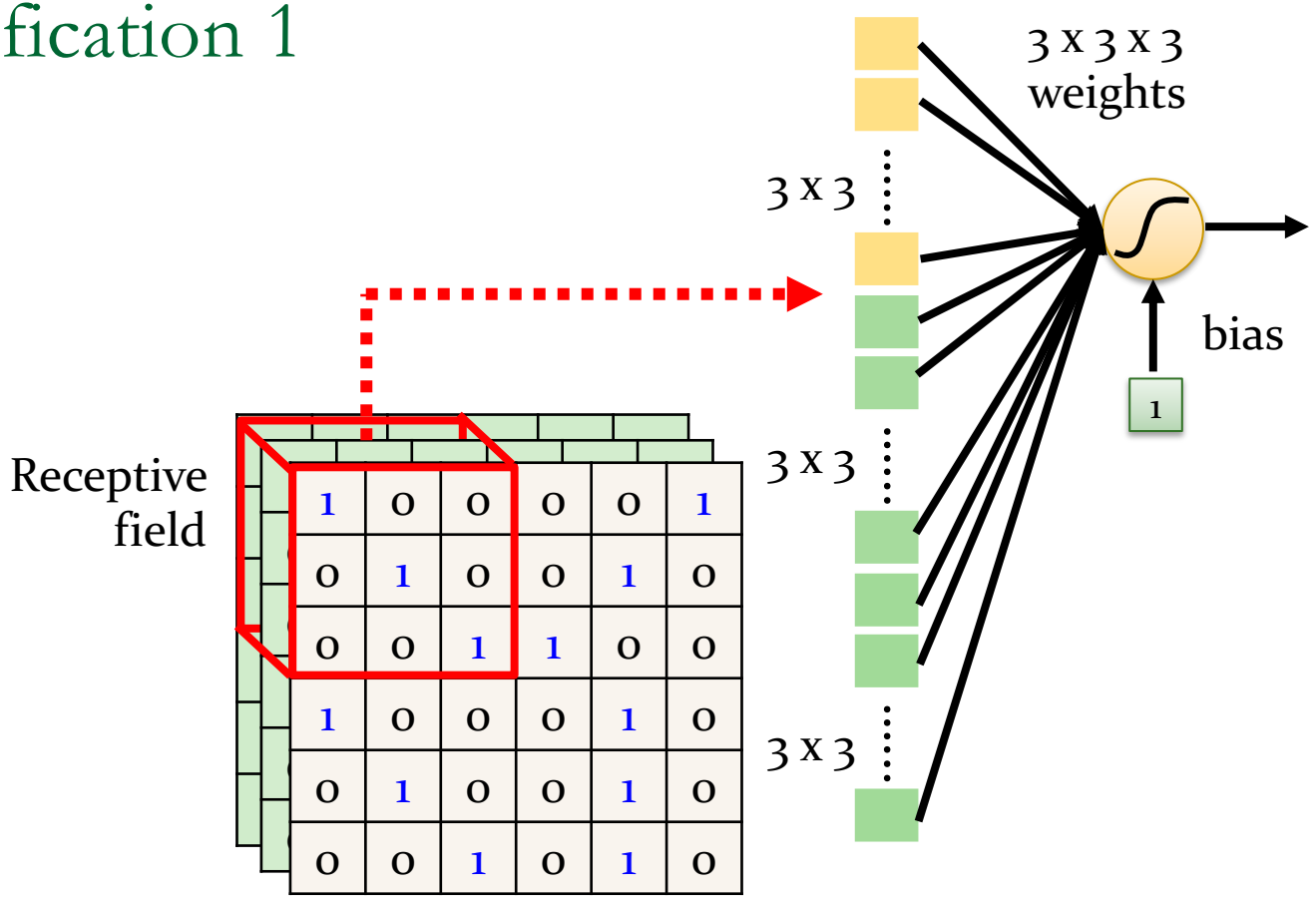
Need to see the whole image?





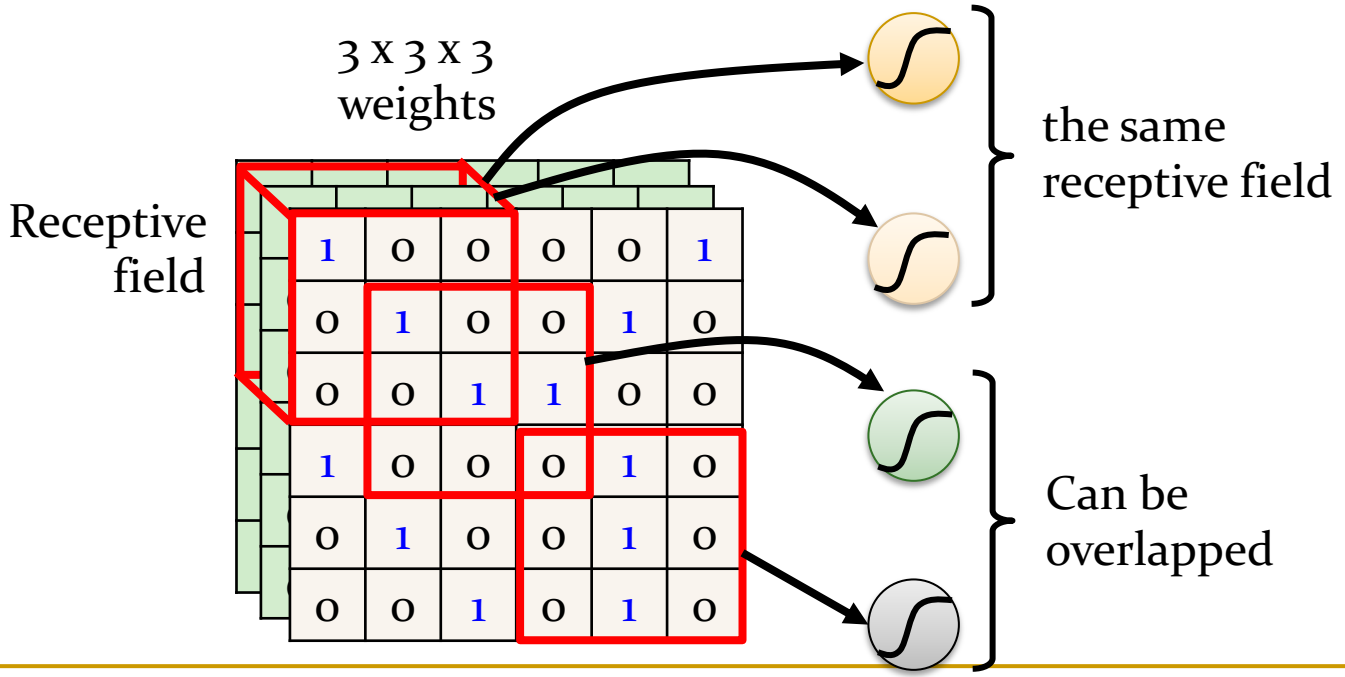
Some patterns are much smaller than the whole image.

Simplification 1



Simplification 1

- Can different neurons have different sizes of receptive field?
- Cover only some channels?
- Not square receptive field?



1. Can different neurons have different sizes of receptive field?

Yes. Different branches can use different filter sizes, such as 3×3 and 5×5, to capture information over different regions. Within a standard convolutional layer, however, filters usually have the same spatial size. Deeper layers can also have larger receptive fields in the original image.

2. Can a filter cover only some input channels?

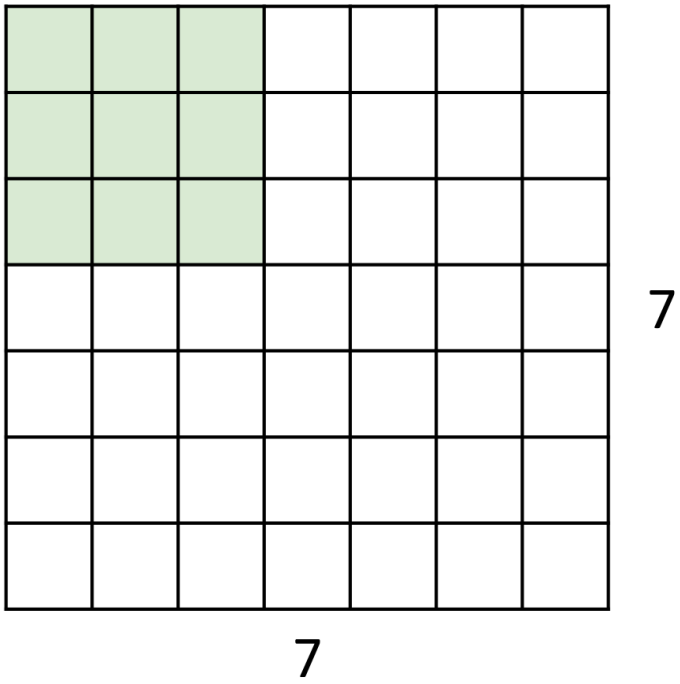
Yes.

- **Standard convolution:** each filter reads all input channels.
- **Grouped convolution:** each filter reads only the channels in its group. For example, splitting 64 channels into four groups gives 16 input channels per filter.
- **Depthwise convolution:** each filter reads only one input channel.

3. Can the receptive field be non-square?

Yes. Filters can be rectangular, such as 3×5, 1×5, or 5×1. A 1×5 filter reads five horizontally adjacent positions, while a 5×1 filter reads five vertically adjacent positions.

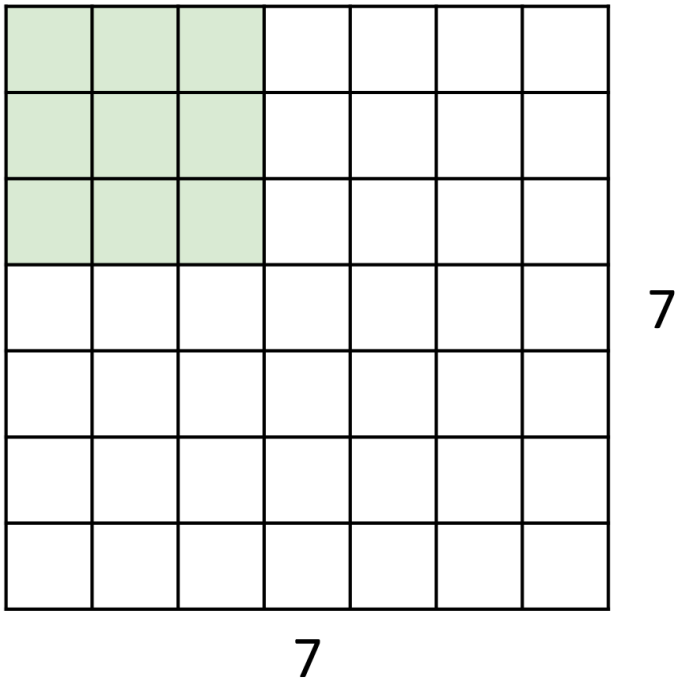
Convolution Spatial Dimensions



Input: 7x7
Filter: 3x3

Q: How big is output?

Convolution Spatial Dimensions

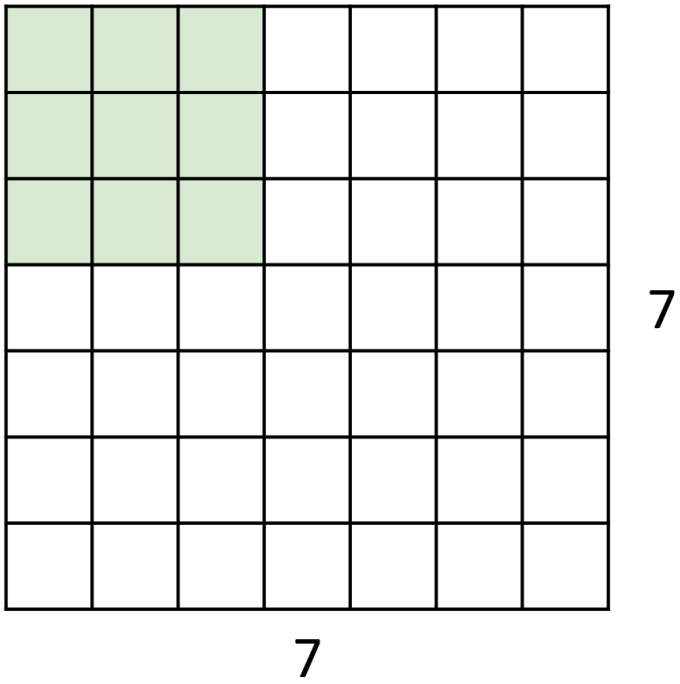


Input: 7x7
Filter: 3x3

Q: How big is output?

Output: 5x5

Convolution Spatial Dimensions



Input: 7x7
Filter: 3x3

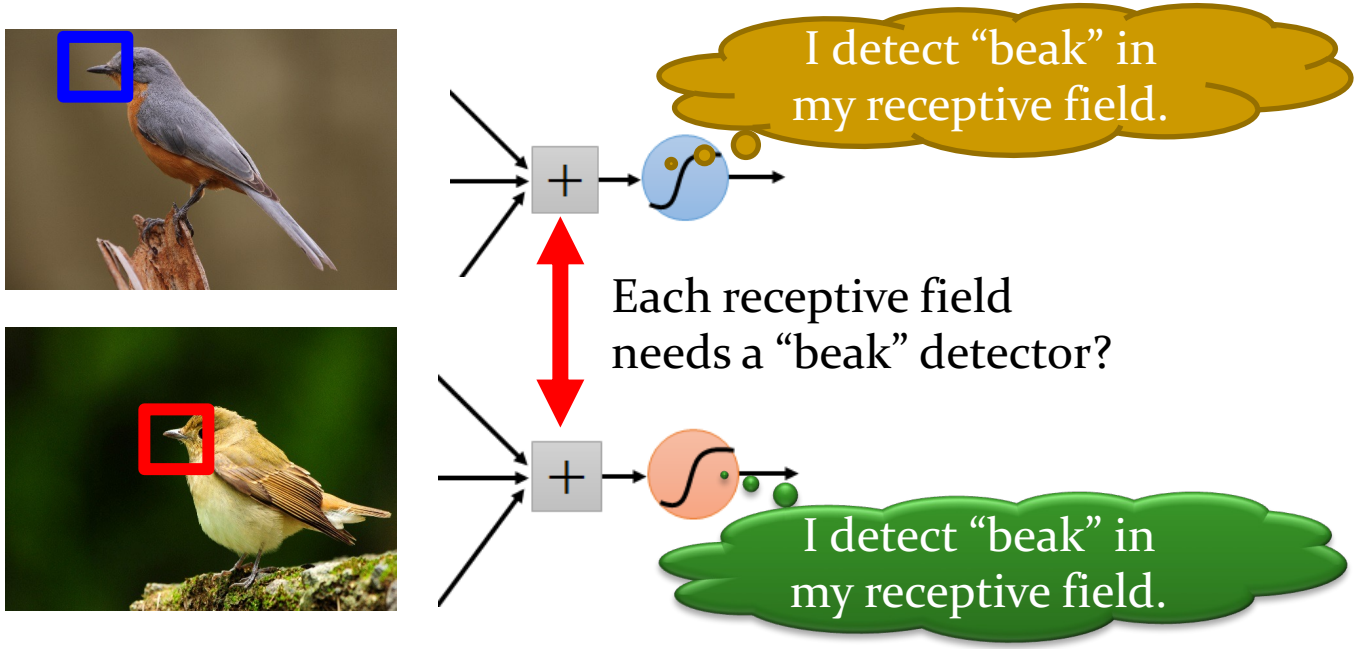
Q: How big is output?

Output: 5x5

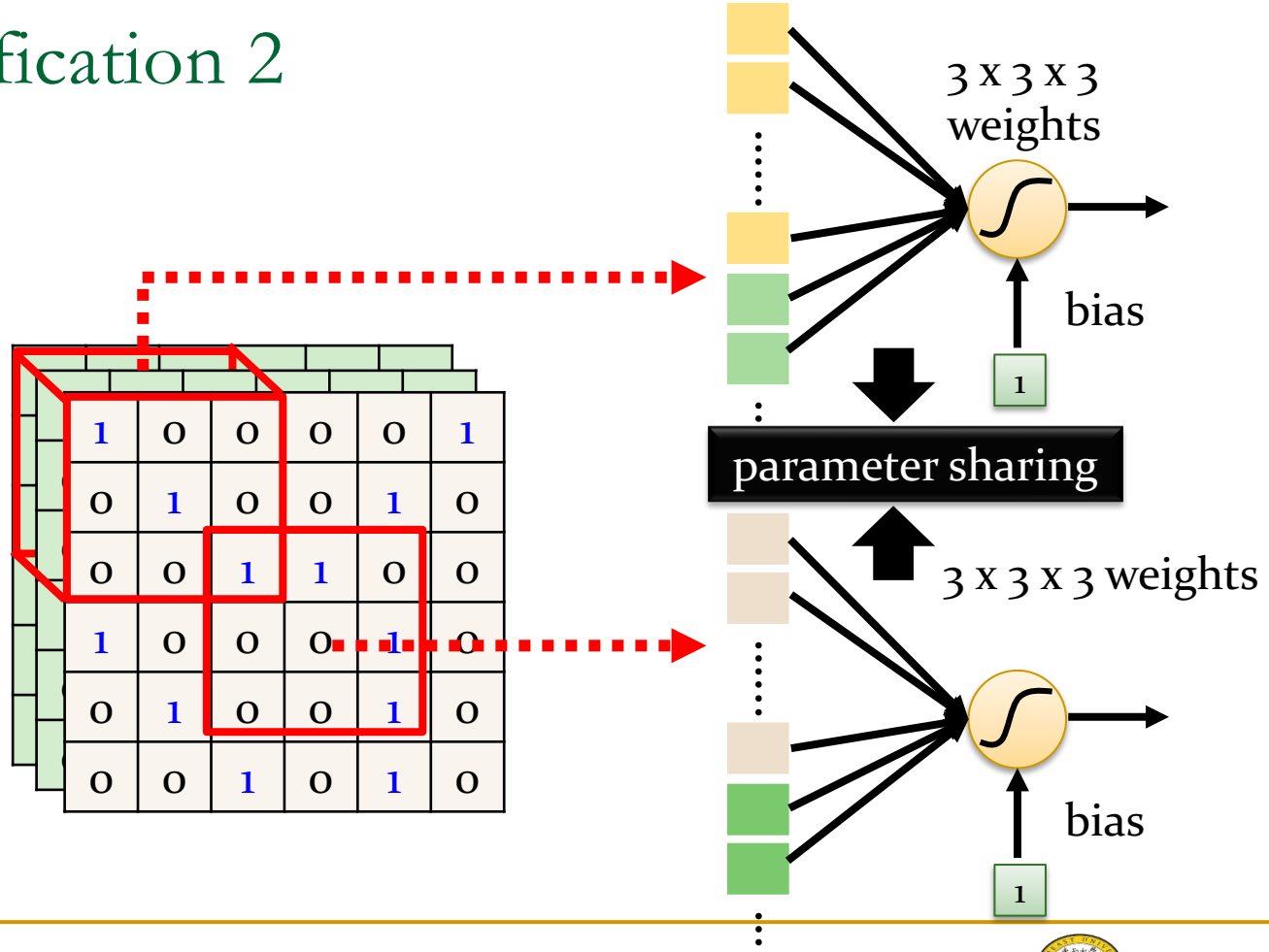
In general:
Input: W **Filter: K**
Padding: P **Stride: S**
Output: floor((W – K + 2P)/S) + 1

Observation 2

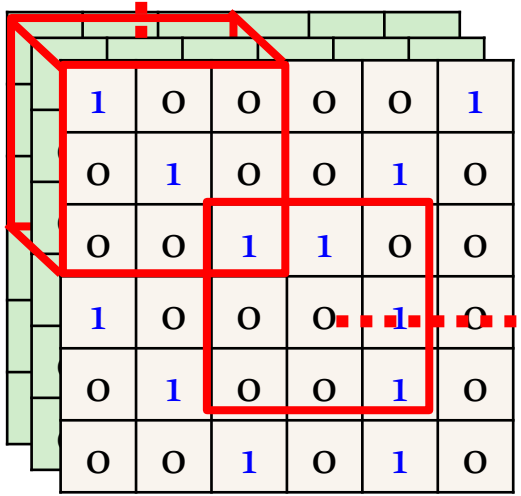
- The same patterns appear in different regions.

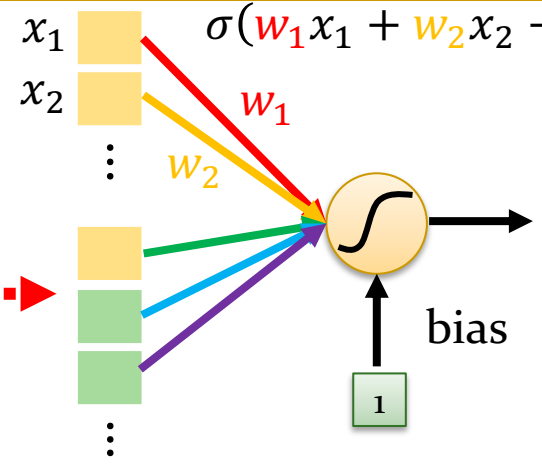


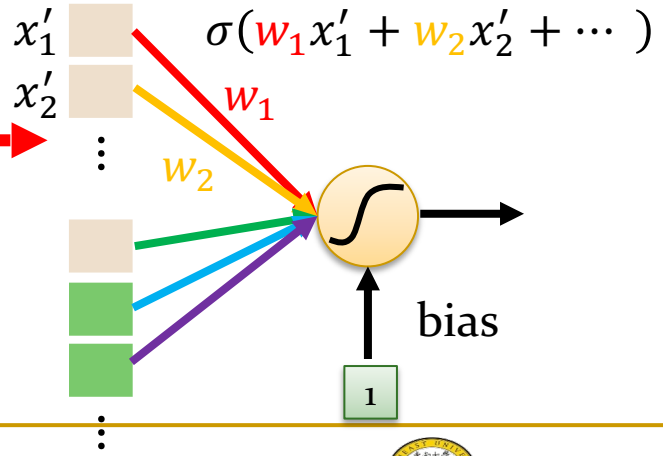
Simplification 2



Simplification 2

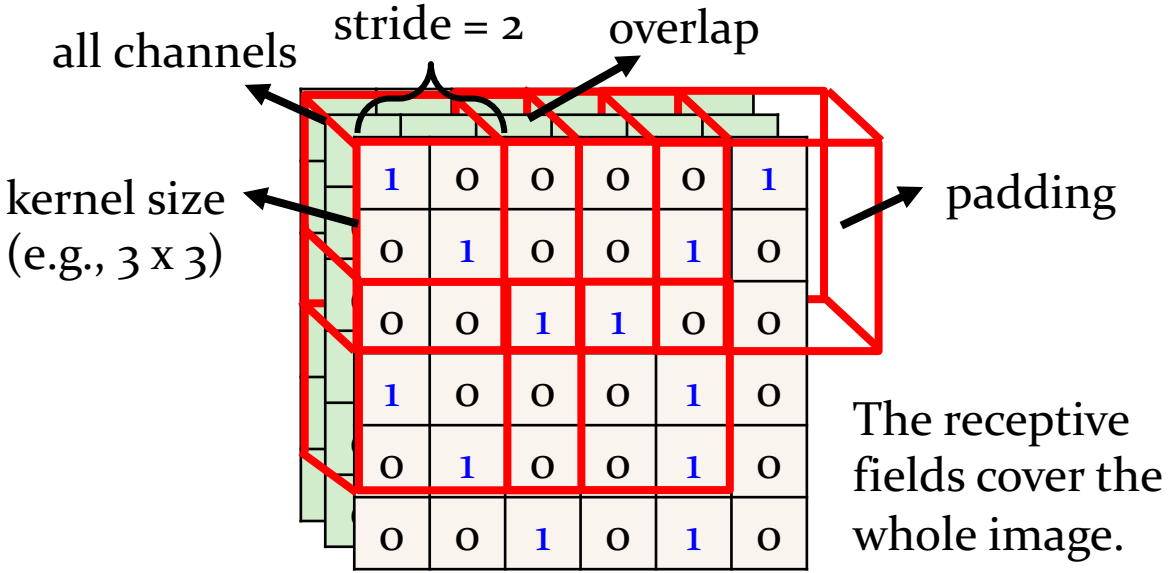






Two neurons with the same receptive field would not share parameters.

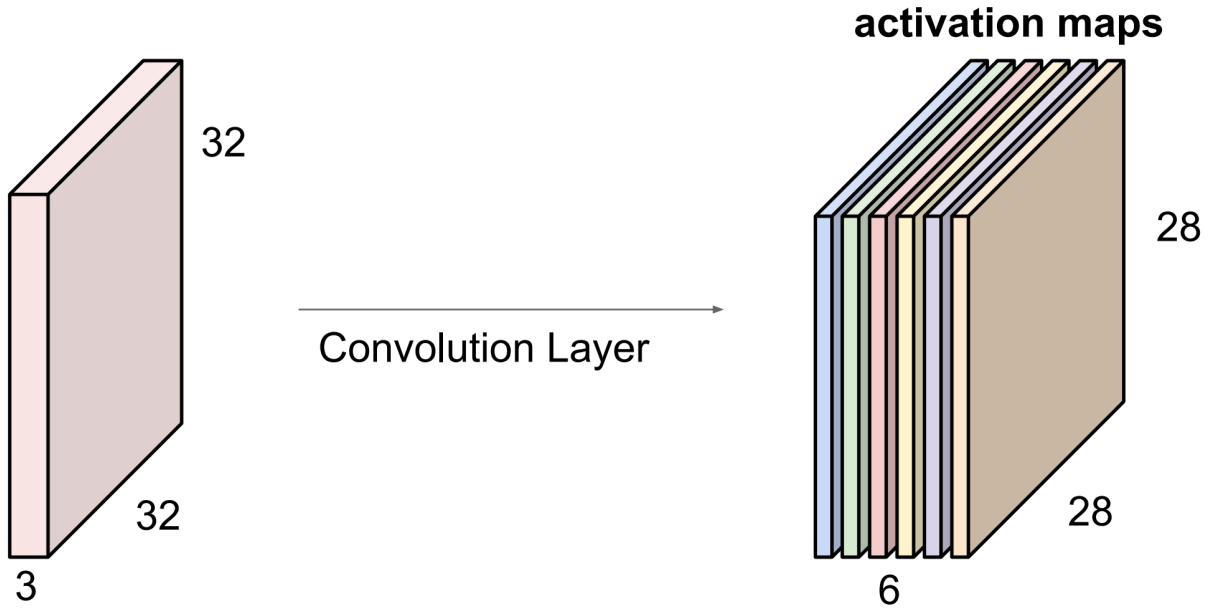
Simplification 2



https://cs231n.github.io/assets/conv-demo/

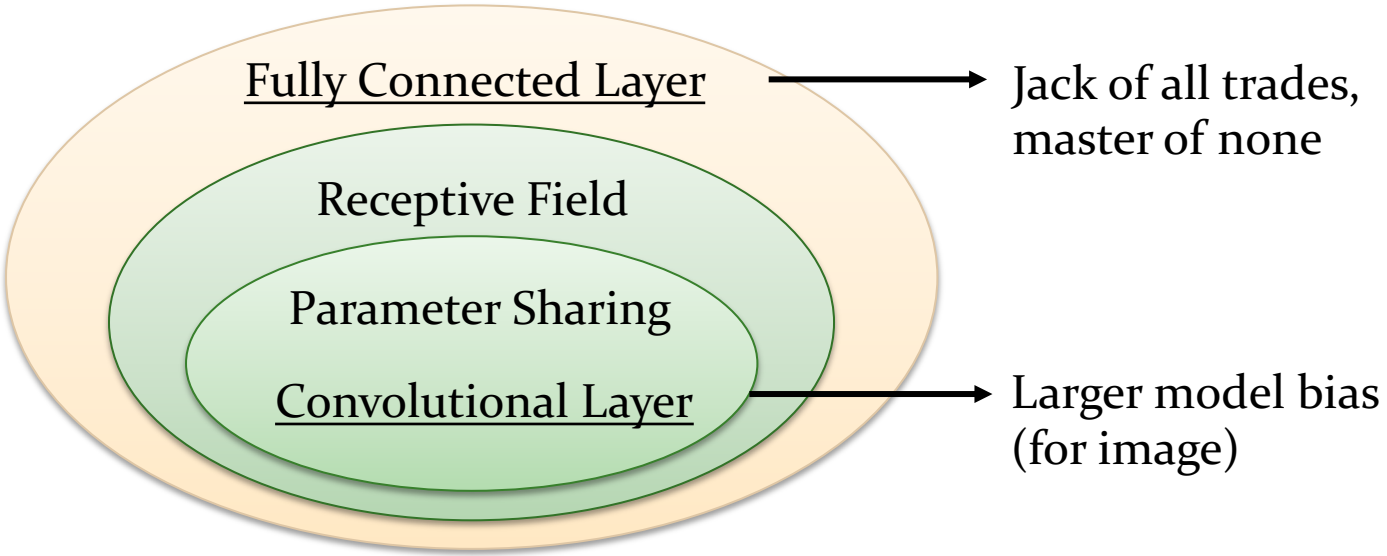
Stacking Convolution Filters

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a "new image" of size 28x28x6!

Benefit of Convolutional Layer



- Some patterns are much smaller than the whole image.
- The same patterns appear in different regions.

Convolutional Layer

Another story based on filter ☺

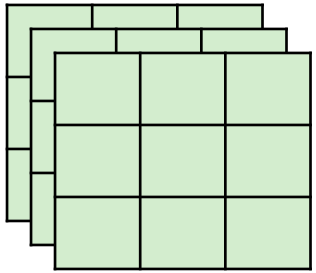


Convolution

channel = 3 (colorful)

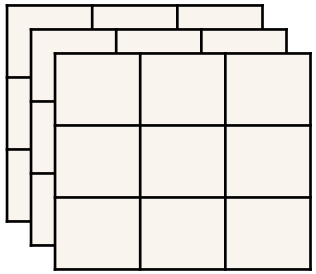
channel = 1 (black and white)

Each filter detects a small pattern (3 x 3 x channel).



Filter 1

3 x 3 x channel tensor

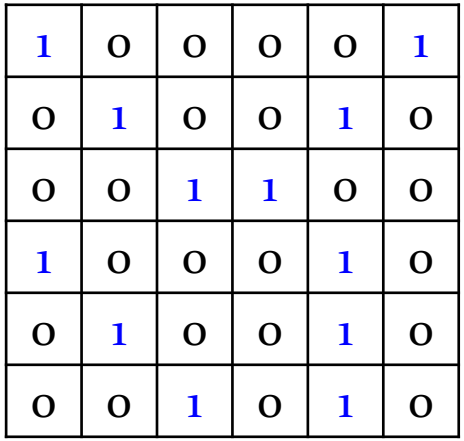


Filter 2

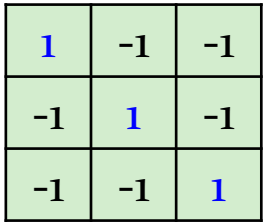
3 x 3 x channel tensor

Convolutional Layer

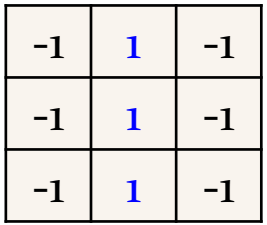
Consider channel = 1
(black and white image)



6 x 6 image



Filter 1



Filter 2

⋮ ⋮

(The values in the filters are unknown parameters.)

Convolutional Layer

-1	1	-1
-1	1	-1
-1	1	-1

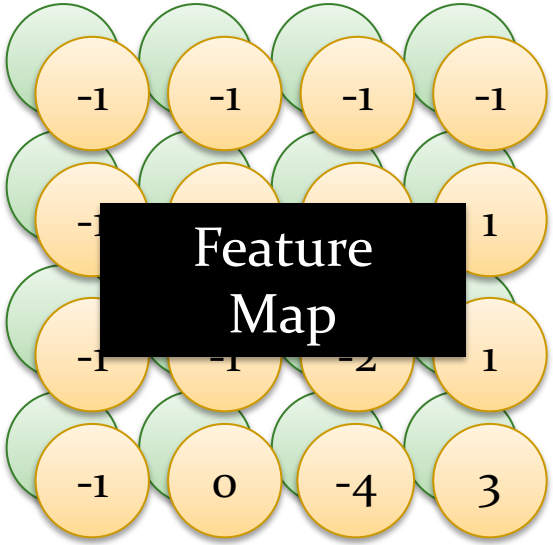
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Do the same process for every filter



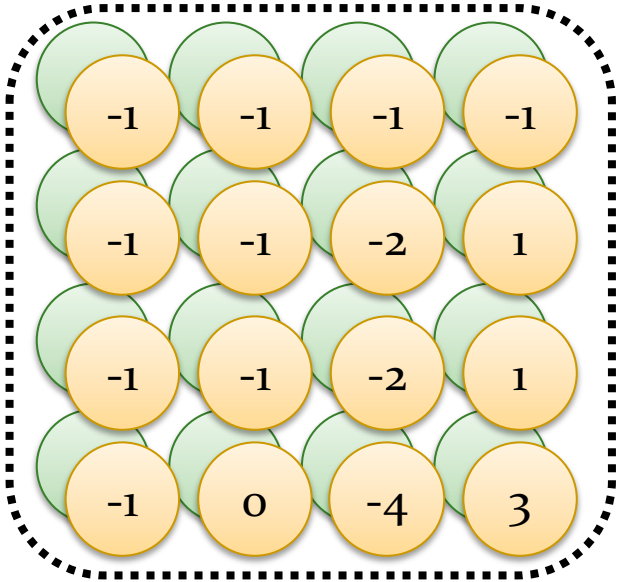
Convolutional Layer



64 filters

Convolution

Convolution



"Image" with 64 channels

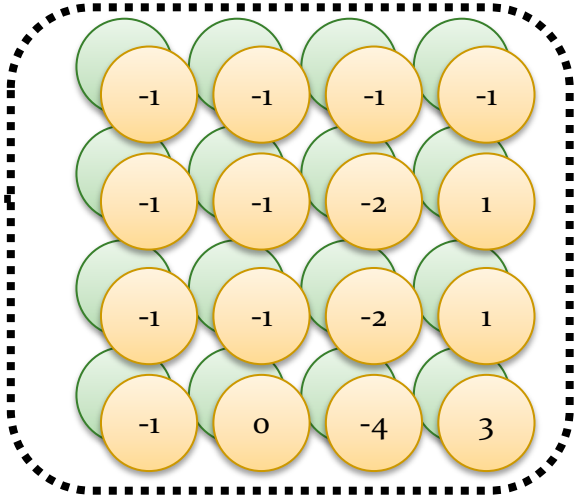
Multiple Convolutional Layer



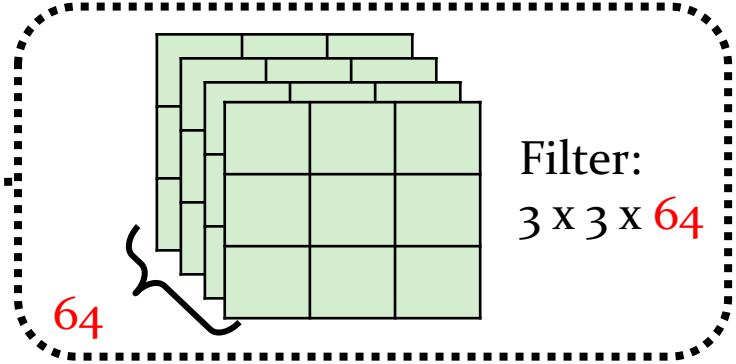
64 filters

Convolution

Convolution



"Image" with 64 channels



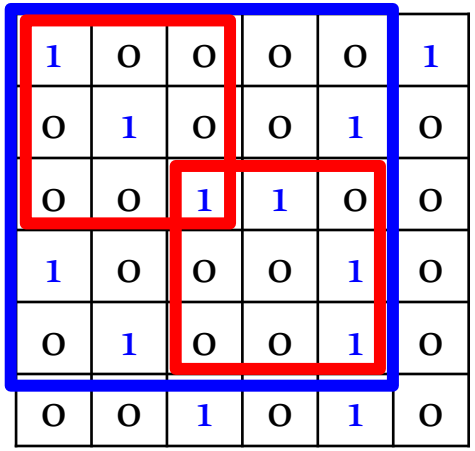
Multiple Convolutional Layer

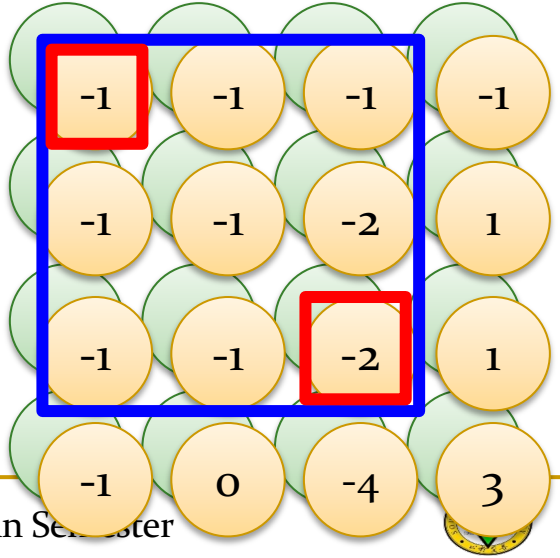


64 filters

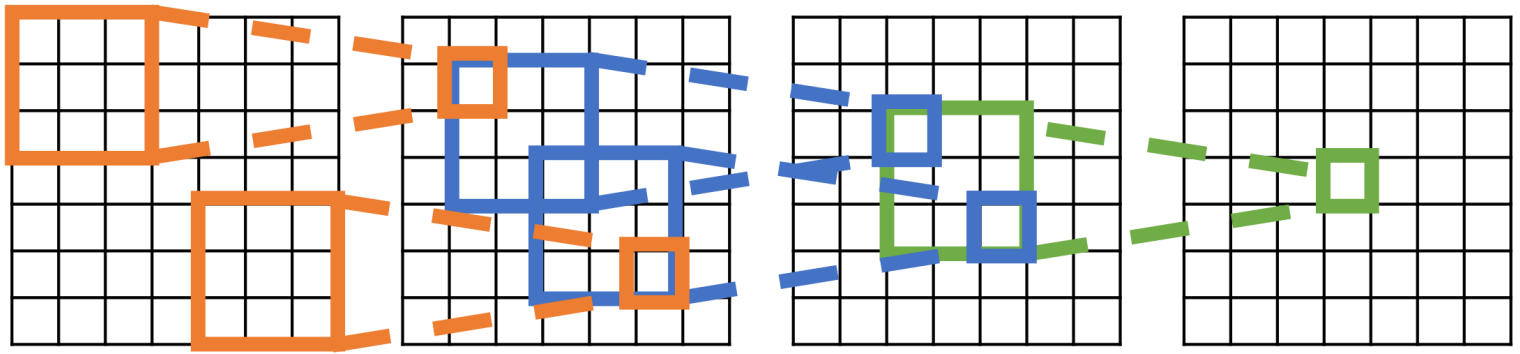
Convolution

Convolution





Receptive Fields



Stack two 3×3 convolutional layers, both with stride 1 and no padding. Place a ReLU between them.

(a) What is the receptive-field size of one second-layer output in the original image: 3×3, 5×5, or 6×6?

(b) Does this mean that the two layers can always be replaced by one 5×5 convolution without changing the function?

Answer.

The receptive field is $\boxed{5 \times 5}$. Along one axis, the second layer reads three adjacent first-layer outputs. Those outputs read original-input positions

$$(1, 2, 3), \qquad (2, 3, 4), \qquad (3, 4, 5).$$

Their union covers five positions. The same argument applies vertically.

No. A single convolution takes a weighted sum of the input values and adds a bias. In the two-layer network, ReLU first sets negative first-layer responses to zero; the second convolution then combines these changed responses. Which responses are set to zero depends on the input, so one convolution with fixed weights generally cannot produce the same result.

Without the intermediate ReLU, the two convolutions in this question could be combined into one 5×5 convolution.

Key point. The same receptive field does not mean the same computation.

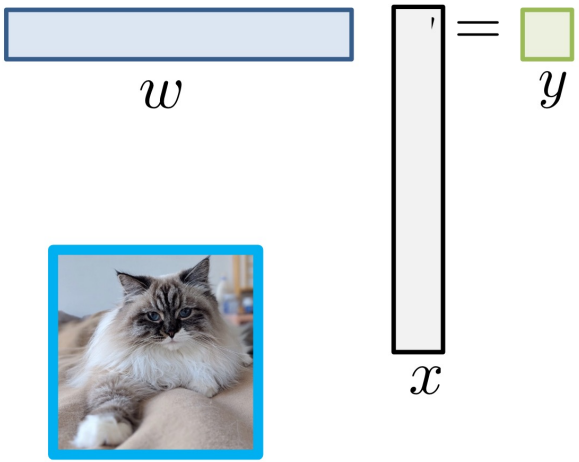
Convolutional Layer

Neuron Version Story	Filter Version Story
Each neuron only considers a receptive field.	There are a set of filters detecting small patterns.
The neurons with different receptive fields share the parameters.	Each filter convolves over the input image.

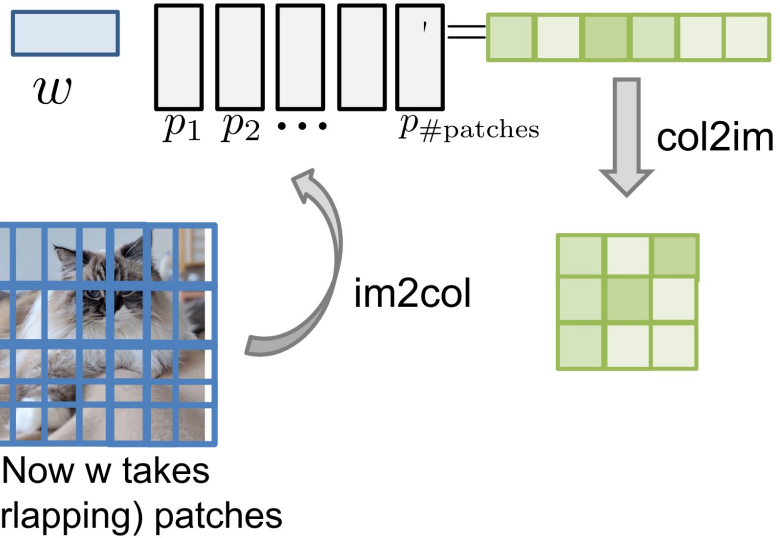
They are the same story.

We're still doing matrix multiplications, just localized & shared

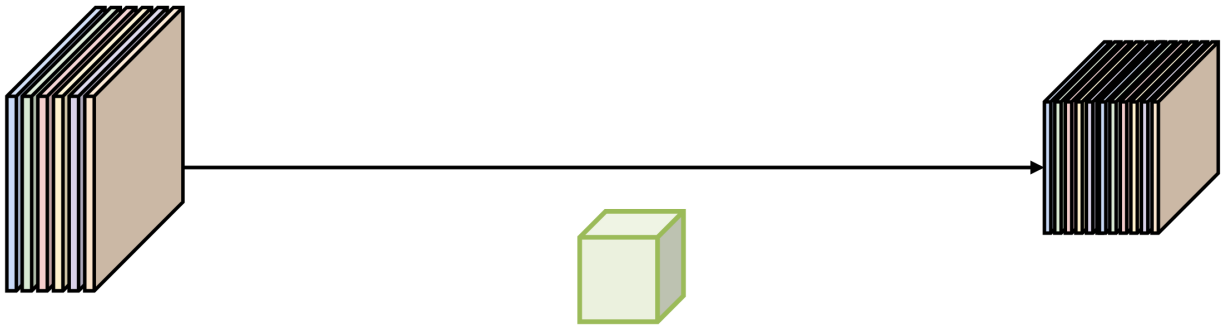
Recall one neuron in FC layer:



With Conv layer:



What needs to be learned?



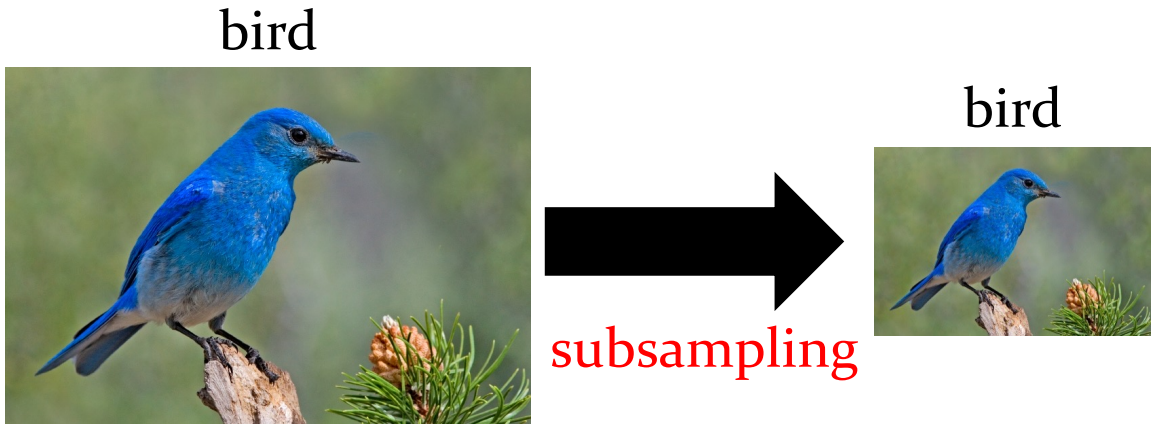
Input:
B, C_in, H, W

W
C_out, C_in, K, K

Output
B, C_out, H_out, W_out

Observation 3

- Subsampling the pixels will not change the object



Pooling – Max Pooling

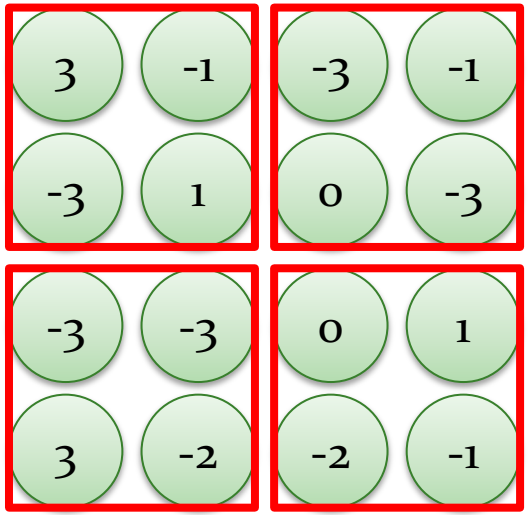
No learnable parameters!

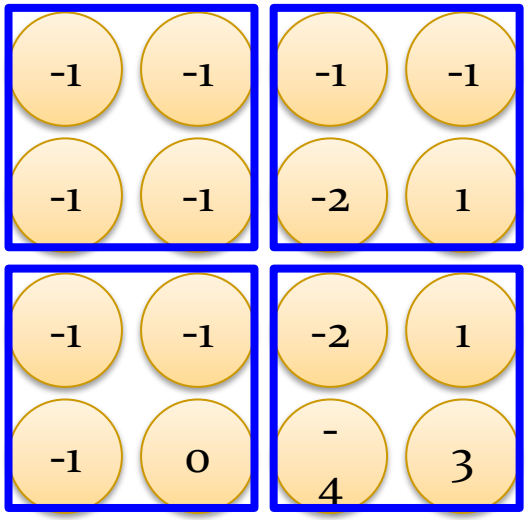
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2





A 2×2 max-pooling operation receives

$$A = \begin{bmatrix} -2 & -1 \\ -3 & -4 \end{bmatrix}.$$

Its scalar output is p, and the incoming derivative is $\partial J / \partial p = 2$. Find p and the gradient with respect to all four entries of A. How many trainable parameters does this pooling operation have?

Answer.

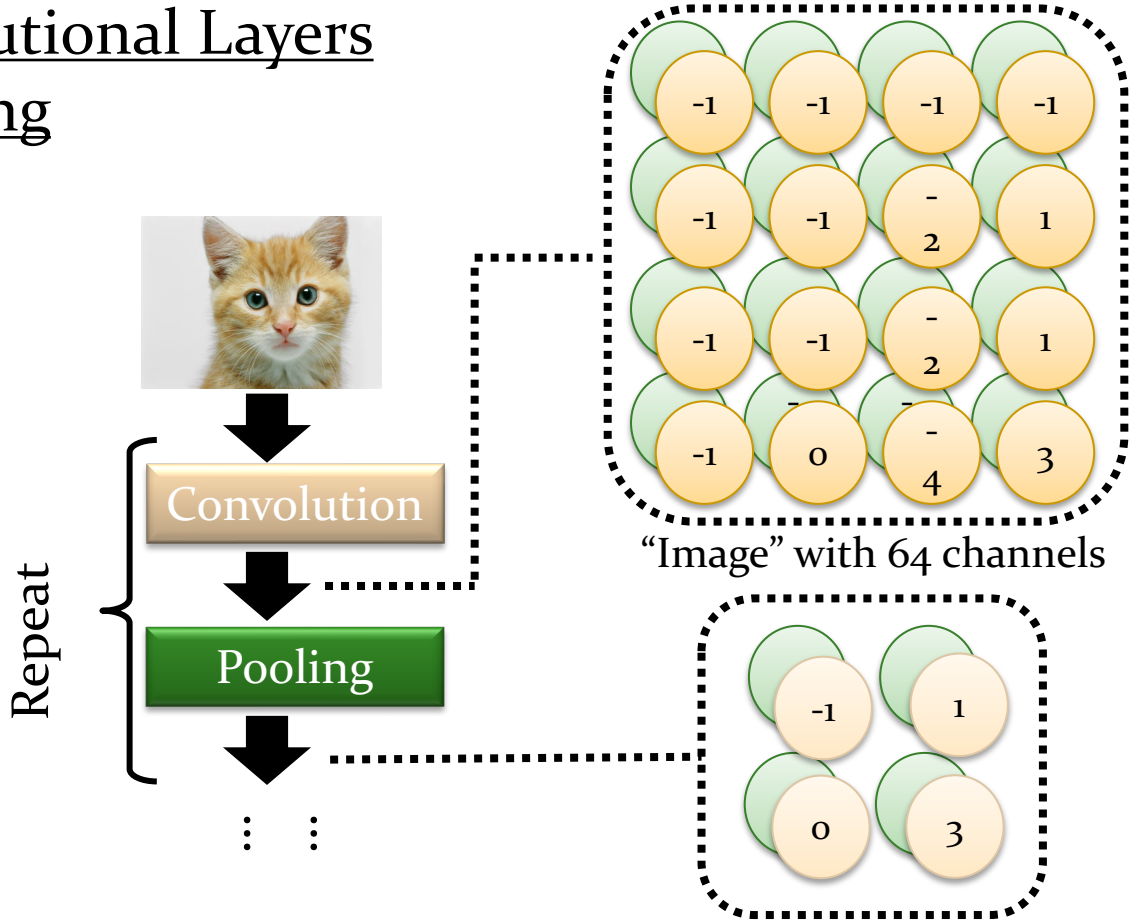
The unique maximum is -1, so

$$p = \boxed{-1}, \qquad \left[\frac{\partial J}{\partial A_{ij}}\right]_{i,j} = \boxed{\begin{bmatrix} 0 & 2 \\ 0 & 0 \end{bmatrix}}.$$

Near this input, only the winning entry changes the maximum. It receives the incoming derivative; the other entries receive zero. There are $\boxed{0}$ trainable parameters. Max pooling does not apply ReLU; its output can be negative.

Key point. Pooling has no weights to update, but its input derivatives are needed to train the earlier convolutional layers.

Convolutional Layers + Pooling



The whole CNNs



Convolution

Pooling

Convolution

Pooling

Flatten

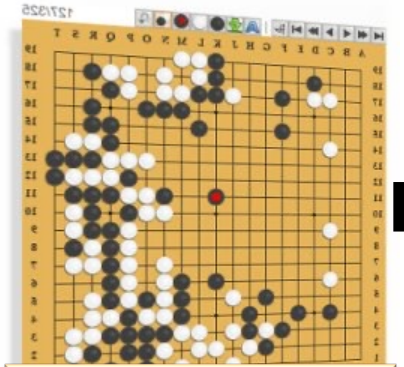
Fully Connected Layers

Softmax

cat dog .....

(they still need to go through non-linearity!!)

Application: Playing Go



19 × 19 matrix (image)

Network

Next move
(19 × 19 positions)
19 × 19 classes

48 channels in Alpha Go

Black: 1
white: −1
none: 0

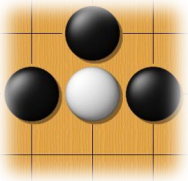
Fully-connected network can be used

But CNN performs much better.

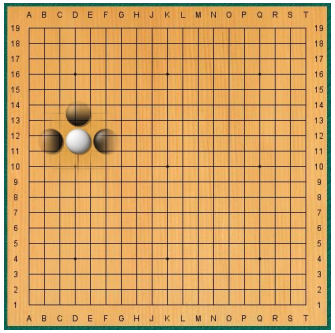
Why CNN for Go playing?

- Some patterns are much smaller than the whole image

Alpha Go uses 5 × 5 for first layer



- The same patterns appear in different regions.



```
import torch.nn as nn
import torch.nn.functional as F


class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x


net = Net()
```

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

```
for epoch in range(2):  # 多批次循环


    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # 获取输入
        inputs, labels = data

        # 梯度置0
        optimizer.zero_grad()

        # 正向传播，反向传播，优化
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

To learn more …

- CNN is not invariant to scaling and rotation (we need data augmentation ☺).







Jürgen Schmidhuber (28 Sep 2025)
Pronounce: You_again Shmidhoobuh
Technical Report IDSIA-09-25, IDSIA

AI Blog
@SchmidhuberAI
juergen@idsia.ch

Who invented deep residual learning?

Modern AI is based on deep artificial neural networks (NNs).[DLH] As of 2025, the most cited scientific article of the 21st century is an NN paper on *deep residual learning* with *residual connections*.[MOST25,25b] Who invented this? Here is the timeline of the evolution of deep residual learning:

课堂练习题

假设一个简单的CNN用于处理MNIST手写数字数据集（输入为28×28像素的灰度图像，通道数为1）

网络结构如下：

1. **卷积层1**：16个3×3卷积核，步幅（stride）为1，填充（padding）为1
2. **池化层1**：2×2最大池化，步幅为2，无填充（padding=0）
3. **卷积层2**：32个3×3卷积核，步幅为1，填充为1
4. **池化层2**：2×2最大池化，步幅为2，无填充

课堂练习题

1. 求卷积层1、池化层1、卷积层2、池化层2的输出特征图尺寸（宽度、高度、通道数）

2. 计算卷积层1和卷积层2的参数数量（包括权重和偏置）

3. 若将池化层2的输出特征图展平后送入一个具有10个神经元的全连接层（对应0-9分类），求该全连接层的参数数量

Answer.

Layer	Output shape	Trainable parameters
Conv_1	$28 \times 28 \times 16$	$16(1 \cdot 3 \cdot 3 + 1) = 160$
Pool_1	$14 \times 14 \times 16$	0
Conv_2	$14 \times 14 \times 32$	$32(16 \cdot 3 \cdot 3 + 1) = 4{,}640$
Pool_2	$7 \times 7 \times 32$	0
Flatten	1,568 values	0
Fully connected	10 values	$10(1{,}568 + 1) = 15{,}690$

Total: $\boxed{20{,}490}$ parameters. The fully connected layer has the most: each of its 10 outputs connects to all $32 \cdot 7 \cdot 7 = 1{,}568$ features.

Key point. The second convolution reads *16 input channels*, not the original image's single channel.